

```
1  # Discord Music Bot
2
3  Музыкальный бот для Discord с управлением из Telegram.
4
5  Возможности:
6
7  * Яндекс Музыка (поиск по названию и по ссылке)
8  * Голосовые каналы Discord
9  * Очередь воспроизведения с автопереходом
10 * Пользовательские плейлисты
11 * Регулировка громкости (по серверам)
12 * Режимы повтора
13 * Красивые Embed-сообщения
14 * Управление из Telegram: кнопочная панель и команды (несколько пользователей)
15 * Авто-кик за запретные слова в чате
16
17 ---
18
19 # Структура проекта
20
21 ```text
22 DiscordBot/
23
24 |— bot.py
25 |— player.py
26 |— playlists.py
27
28 |— .env           # токены и настройки (НЕ коммитить)
29 |— .env.example
30 |— requirements.txt
31 |— .gitignore
32
33 |— data/
34 |   |— playlists.json
35 |   |— queue.json
36 |   |— settings.json
37
38 |— model/         # модель Vosk (только для голосового /listen)
39 |— ffmpeg/
40 ```
41
42 ---
43
44 # Установка и запуск
45
46 1. Установить зависимости:
47
48 ```bash
```

```

49     pip install -r requirements.txt
50     ```
51
52     `PyNaCl` обязателен для голоса. `aiogram` — для Telegram. `vosk`/`numpy`/`discord-ext-voice-recv` нужны толь
ко для голосового `/listen`.
53
54 2. Создать `.env` по образцу `.env.example` (см. таблицу переменных ниже).
55
56 3. Положить `ffmpeg.exe` по пути из `FFMPEG_PATH`.
57
58 4. Запустить:
59
60     ```bash
61     python bot.py
62     ```
63
64     Discord и Telegram-бот поднимаются вместе, в одном процессе.
65
66     ---
67
68     # Переменные .env
69
70     ```text
71     DISCORD_TOKEN=...      # токен Discord-бота (обязательно)
72     YANDEX_TOKEN=...       # токен Яндекс Музыка (обязательно)
73     FFMPEG_PATH=.\\ffmpeg\\bin\\ffmpeg.exe  # путь к ffmpeg (без кавычек и без r)
74     GUILD_ID=...          # ID сервера — для мгновенной синхронизации слэш-команд
75
76     TELEGRAM_TOKEN=...     # токен Telegram-бота от @BotFather (опционально)
77     TELEGRAM_ADMIN_ID=...  # твой Telegram ID; несколько — через запятую
78     VOICE_CHANNEL_ID=...   # ID голосового канала для музыки из Telegram
79
80     VOSK_MODEL_PATH=model  # путь к модели Vosk (только для /listen)
81     ```
82
83     * Свой Telegram ID можно узнать у бота `@userinfobot`.
84     * `GUILD_ID` и `VOICE_CHANNEL_ID` копируются правым кликом по серверу/каналу при включённом Режиме разработчика
в Discord.
85
86     ---
87
88     # Discord-команды
89
90     ## Голосовой канал
91
92     * `/join` — подключиться к голосовому каналу.
93     * `/leave` — отключиться.
94

```

```
95  ## Воспроизведение
96
97  * `/play <название|ссылка>` – найти трек и добавить в очередь. Если ничего не играет – запускается сразу; иначе
    встаёт в конец. Бот сам заходит в канал.
98  * `/skip` – пропустить текущий трек.
99  * `/pause` – пауза.
100 * `/resume` – продолжить.
101 * `/stop` – остановить и очистить очередь.
102 * `/volume <0-200>` – громкость (сохраняется для сервера).
103 * `/nowplaying` – текущий трек.
104
105  ## Очередь
106
107  * `/queue` – текущий трек и очередь.
108  * `/clear` – очистить очередь.
109  * `/shuffle` – перемешать.
110  * `/loop off|track|queue` – режим повтора.
111
112  ## Плейлисты
113
114  * `/playlists` – список своих плейлистов.
115  * `/playlist_create <название>` – создать.
116  * `/playlist_show <название>` – показать содержимое.
117  * `/playlist_delete <название>` – удалить.
118  * `/playlist_add <название> <трек|ссылка>` – добавить трек.
119  * `/playlist_remove <название> <номер>` – удалить трек по номеру.
120  * `/playlist_play <название>` – добавить плейлист в очередь и запустить.
121  * `/playlist_rename <старое> <новое>` – переименовать.
122
123  ---
124
125  # Авто-кик за слова
126
127  Кто напишет в чат «Кирилл» или «Рыжий» (в любом регистре) – бот кикает автора.
128
129  Требования:
130
131  * В Developer Portal включён Message Content Intent.
132  * У роли бота есть право Kick Members, и роль бота стоит выше кикаемого (владельца сервера кикнуть нель-
    зя).
133
134  Список слов – в `KICK_TRIGGERS` в начале `bot.py`.
135
136  ---
137
138  # Управление из Telegram
139
```

```
140 Бот поднимает Telegram-управление, если задан `TELEGRAM_TOKEN`. Доступ — только у пользователей из `TELEGRAM_AD
141 MIN_ID` (несколько через запятую), остальных бот молча игнорирует. Это касается и команд, и нажатий кнопок.
142 Музыка из Telegram играет в канал `VOICE_CHANNEL_ID`.
143
144 ## Кнопочная панель (основной способ)
145
146 `/start` или `/menu` — открывает пульт с кнопками. Сверху видно: что играет, сколько в очереди, громкость и реж
147 им повтора. После каждого нажатия панель обновляется.
148
149 Кнопки: II Пауза, ► Продолжить, ⏭ Пропустить, ■ Стоп, 🔄 Перемешать, 🧹 Очистить, 🔊 -10 / 🔊 +10 (громкост
150 ь шагом 10%), 🔄 Повтор (по кругу off→track→queue), 📁 Очередь, 📁 Плейлисты, 🔄 Обновить.
151
152 **Добавить трек** — просто пришли его название или ссылку обычным сообщением (без команды). Бот добавит трек в
153 очередь и покажет панель.
154
155 **Плейлисты** — кнопка «📁 Плейлисты» показывает список плейлистов кнопками; тап проигрывает весь плейлист.
156 «⬅ Назад» — в меню.
157
158 ## Команды (тоже работают)
159
160 Музыка: `/play <название|ссылка>`, `/skip`, `/pause`, `/resume`, `/stop`, `/volume <0-200>`, `/queue`, `/np`,
161 `/clear`, `/shuffle`, `/loop off|track|queue`, `/join`, `/leave`.
162
163 Плейлисты: `/playlists`, `/playlist_create <название>`, `/playlist_show <название>`, `/playlist_delete <названи
164 е>`, `/playlist_add <название> | <трек|ссылка>`, `/playlist_remove <название> <номер>`, `/playlist_play <назван
165 ие>`, `/playlist_rename <старое> | <новое>`.
166
167 `/help` — текстовая справка по командам.
168
169 Особенности формата (Telegram передаёт аргументы одной строкой):
170
171 * `/playlist_add` и `/playlist_rename` используют разделитель `|`.
172 * `/playlist_remove` берёт номер последним словом, остальное — название.
173
174 > Плейлисты в Telegram у каждого пользователя свои (ключ `tg:<id>`) и отделены от плейлистов в Discord — чтобы
175 при нескольких людях списки не смешивались. Очередь и плеер при этом общие на сервере.
176
177 ---
178
179 # Хранение данных
180
181 ## data/queue.json
182
183 ```json
184 {
185   "987654321": {
186     "queue": [
```

```
179     {
180         "title": "Numb",
181         "artist": "Linkin Park",
182         "query": "Linkin Park Numb",
183         "track_id": "12345678"
184     }
185 ],
186 "loop": "off"
187 }
188 }
189 ```
190
191 ## data/playlists.json
192
193 ```json
194 {
195     "123456789": {
196         "rock": [
197             {
198                 "title": "Numb",
199                 "artist": "Linkin Park",
200                 "query": "Linkin Park Numb",
201                 "track_id": "12345678"
202             }
203         ]
204     }
205 }
206 ```
207
208 ## data/settings.json
209
210 ```json
211 {
212     "987654321": {
213         "volume": 1.0,
214         "now_playing": null
215     }
216 }
217 ```
218
219 ---
220
221 # Что работает
222
223 * [x] Воспроизведение через Яндекс Музыку (название и ссылки)
224 * [x] Очередь с автопереходом и режимами повтора
225 * [x] Пауза / продолжение / стоп / пропуск / громкость
226 * [x] Плейлисты: создание, просмотр, удаление, добавление/удаление треков, проигрывание, переименование
```

```
227 * [x] Авто-подключение и авто-отключение из пустого канала
228 * [x] Embed-сообщения
229 * [x] Авто-кик за слова в текстовом чате
230 * [x] Управление из Telegram: кнопочная панель + команды (несколько пользователей)
231
232 # Что не работает / опционально
233
234 * [ ] Голосовое распознавание (`/listen`, `/stoplisten`) – **недоступно**: с марта 2026 Discord включил обяза-
льное сквозное шифрование голоса (DAVE), и бот не может расшифровать аудио участников. Код оставлен, но фича уп-
ирается в шифрование.
235 * [ ] Поддержка VK Музыки – не реализована.
236 * [ ] `/remove` для удаления трека из очереди по номеру – не реализована (функция в коде есть).
237
238 ---
239
240 # Технические заметки
241
242 * Discord и Telegram-бот работают в одном процессе на общем asyncio-цикле: `asyncio.gather(client.start(...), d
p.start_polling(...))`. Telegram-хендлеры дёргают те же функции плеера, что и слэш-команды.
243 * Прямые ссылки Яндекса протухают, поэтому в очереди и плейлистах хранится `track_id` + `query`, а ссылка на по-
ток получается заново перед воспроизведением.
244 * Блокирующие запросы к Яндексу выполняются в отдельном потоке (`asyncio.to_thread`), чтобы бот не подвисал.
245 * Автопереход на следующий трек – через `after`-колбэк FFmpeg с пробросом корутины в event loop.
246 * Токены берутся только из `.env`; файл в `.gitignore`.
247 * Слэш-команды синхронизируются в `GUILD_ID` (мгновенно). Без него – глобально, с задержкой до часа.
248 * Требуется Python 3.13.4+ – в более ранних 3.13 при старте возможно зависание на медленном WMI-вызове внутри `
platform` (баг самого Python).
249
```